LAB 5

Modeling and Testing of Memory Module and Memory Interface

Engr. Ali Sabir
Dr. Haroon Waris

Electrical Engineering
Department National
University of Computer &
Emerging Sciences (FAST-NU)
Islamabad-44000, Pakistan

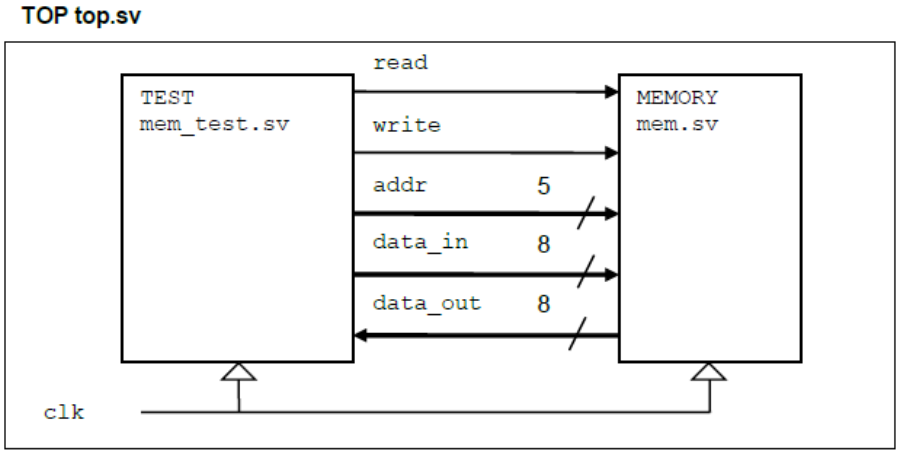
FAST-NUCES

Objective

- To use SystemVerilog subprogram enhancements to test a memory module.

Create the stimulus task using System Verilog subprogram enhancement and verify the supplied memory design.

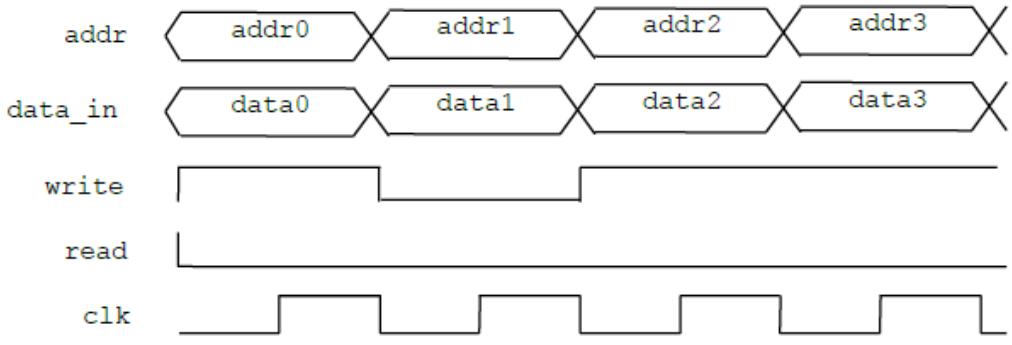
Read the specification first and then follow the instruction in the below statement and completing the Memory Testbench.



Specification

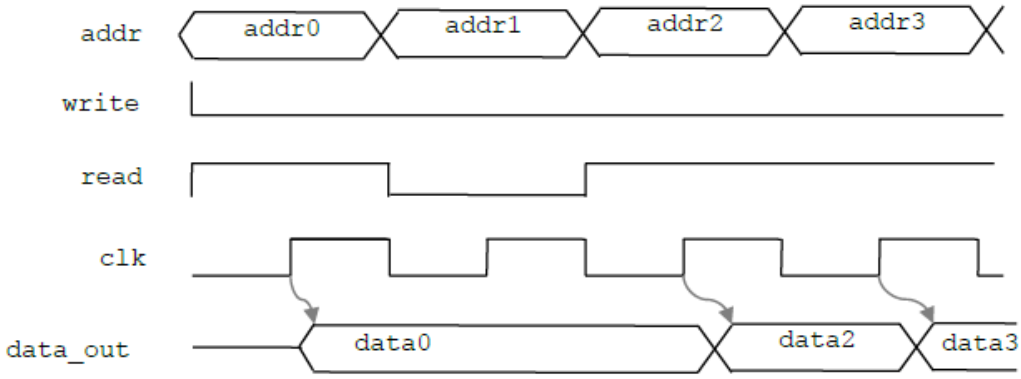
- **addr** is a 5-bit logic vector. **data_in** and **data_out** are both 8-bit logic vectors. **read**, **write** and **clk** are logic.
- Memory write**: data_in** is written in the **memory[addr]** on the positive edge of the **clk** when **write=1**

Memory Write Cycle



- Memory read: **data_out** is assigned from **memory[addr]** on the positive edge of **clk** when **read=1**

Memory Read Cycle



- **read** and **write** should never be simultaneously high

Completing the Memory Testbench

Working in the lab06-mem directory, perform the following

1. The memory design is already being written (mem.sv) you are going to complete the memory testbench in the mem_test.sv
   You will be using the following system verilog construct
   - Void function
   - Argument passing by name (.name(name))
   - Default formal argument with default values
   - Output should match to the below Console output

3. Declare a task (`read_mem`) to read the memory as follows:

   a) Define an input argument for the address value and an output argument for the read data. Remember to use blocking assignment for assigning read data to output argument, so that assignment is complete upon return to the caller.
   b) Assign `addr` from the argument, and drive `read` and `write`, synchronized to the clock. Hint: drive signals on the inactive clock edge to avoid race conditions.
   c) Assign the output data argument from `data_out` at an appropriate time. There is a short propagation delay between the rising edge of `clk` and `data_out` updating.
   d) Input argument `debug` with a default value of 0. If debug = 1, display the read address and data values.

4. Complete the "Clearing the Memory" test by writing zero to every address location, and then reading back and checking the data read matches the data written.

5. Complete the "Data = Address" test by writing data equal to the address to

FAST NUCES

every address location, and then reading back and checking the data read matches the data written.

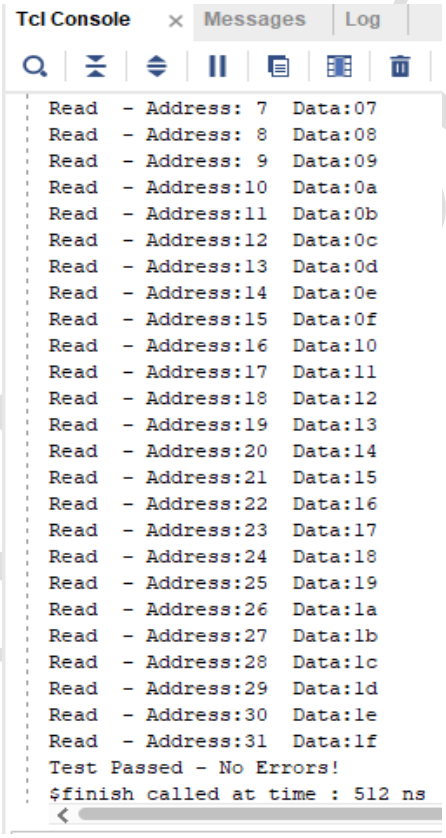
6. Write a `void` function (`printstatus`) with an input argument `status` which indicates the number of errors encountered in the tests. If `status` = 0, the test passed.

Testing the Memory

7.Simulate the testbench and memory design.

Remember to compile also the `top.sv` module, which instantiates and connects both design and testbench.

Debug your subprograms as required until you are happy the design is verified.



Assignment

To use a SystemVerilog interface with ports, modports and methods. Modify the Memory testbench and design to connect via a SystemVerilog interface.

Adding the Memory Interface

Working in the lab07-intf directory, perform the following.

1. Copy your Memory design (mem.sv), testbench (mem_test.sv) and top-level module (top.sv) from lab06-mem into lab07-intf.
2. Define the Memory interface in a new file and declarations for the addr, data_in, data_out, read and write signals.
3. Edit your Memory design and testbench by updating the port list with an interface port and referencing the interface signals via the interface port name.
4. Modify the top-level module to make an instantiation of the interface and connect this to the Memory design and testbench instances.
5. Rerun your memory test to check that the interface is working correctly.
6. Add a clock input port to your interface. Remove the clock ports from your Memory design and testbench modules. Update your interface instance to map the clk signal to the clock port. Rerun your memory test to check the interface port is working correctly.
7. Add modports to the interface to define directional information for both the design and testbench. Reference the modports in your design and testbench port list. Rerun your memory test to check that the modports work correctly.
8. Redefine the write_mem() and read_mem() tasks as interface methods:
    a. Move the task declarations into the interface.
    b. Update the testbench to reference the tasks via the interface.
    c. Update your testbench modport to allow access to the interface methods via an import statement.
    d. Rerun your memory test.

FAST-NICES